

预处理命令

复习

- | | |
|-----------------|----|
| 1. 结构体概述 | 了解 |
| 2. 结构体的定义和初始化 | 重点 |
| 3. 结构体指针 | 重点 |
| 4. 结构体数组 | 重点 |
| 5. 结构体做函数参数 | 重点 |
| 6. 共用体 | 熟悉 |
| 7. 枚举类型 | 熟悉 |
| 8. 用typedef定义类型 | 熟悉 |

目录

- | | |
|------------|----|
| 1. 预处理概述 | 了解 |
| 2. 常用预处理指令 | 熟悉 |
| 3. 宏定义 | 重点 |
| 4. 宏函数 | 重点 |
| 5. 取消宏定义 | 重点 |
| 6. 条件编译 | 重点 |

预处理概述

预处理是 C++ 的一个重要功能，预理由预处理程序负责完成。

预处理是指在进行编译的第一遍扫描（词法扫描和语法分析）之前所作的工作。

当对一个源文件进行编译时，系统把自动引用预处理程序对源程序中的预处理部分作处理，处理完毕自动进入对源程序的编译。

C++ 提供了多种预处理功能，例如宏定义、文件包含、条件编译等。合理地使用预处理功能编写的程序便于阅读、修改、移植和调试，还有就是利于模块化程序设计。

预处理概述

`#include <iostream>` 其实就是一个预处理命令

预处理主要是处理以 `#` 开头的命令。

预处理命令要放在所有函数之外，而且一般都放在源文件的前面。

常用预处理指令

文件包含类

`#include` 包含一个源代码文件，头文件

宏定义类

`#define` 定义宏

`#undef` 取消已定义的宏

条件编译类

`#if` 如果给定条件为真，则编译下面代码

`#ifdef` 如果宏已经定义，则编译下面代码

`#ifndef` 如果宏没有定义，则编译下面代码

`#elif` 如果`#if`条件不为真,当前条件为真，则编译下面代码

`#endif` 结束`#if...到 #else`条件编译块

宏定义

C++ 源程序中允许用一个标识符来表示一个字符串，称为“宏”。被定义为宏的标识符称为“宏名”。

在编译预处理时，对程序中所有出现的宏名，都用宏定义中的字符串去代换，这称为宏替换或宏展开。

宏定义是由源程序中的宏定义命令完成的。

宏替换是由预处理程序自动完成的。在 C 语言中，宏定义分为有参数和无参数两种。

宏定义

无参数宏定义

语法

#define identify expression

#define	定义宏使用的关键字。
identify	宏的标识符，也就是宏名。
expression	该宏所代表的表达式或者常量值。

案例：

```
#define PI 3.14
```

定义了一个宏 PI，其代表的常量值为 3.14，以后，我们需要使用 3.14 的地方，可以直接使用 PI 来代替。

宏定义案例

利用宏定义PI，输入半径r，完成求圆面积的代码。

宏定义案例 代码

```
#define PI 3.1415926
```

```
double r;
```

```
cin >> r;
```

```
cout << PI *r * r << endl;
```

宏定义

带参数宏定义

语法

#define identify(paramlist) expression

#define	定义宏使用的关键字。
identify	宏的标识符，也就是宏名。
paramlist	宏的参数列表。
expression	该宏所代表的表达式或者常量值。

宏定义

案例：

```
#define INC(x) x++
```

定义了一个宏 INC，该宏接受一个参数 x，该宏实现的功能是将当前 x 的值自增

带参宏定义案例

利用宏定义简化num变量自增，自减的代码

带参宏定义案例

```
#define INC ( v ) v++
```

```
#define DEC ( v ) v--
```

```
int num= 100 ;
```

```
cout << "num = " << num << endl;
```

```
INC(num);
```

```
cout << "num = " << num << endl;
```

```
DEC(num);
```

```
cout << "num = " << num << endl;
```

带参宏定义

```
#define SQR(n) n*n
```

```
int num = 10;
```

```
int result = SQR(num);
```

```
cout << result << endl;
```

```
int result2 = SQR(5+5);
```

```
cout << result2 << endl;
```

输出一样吗？

带参宏定义

```
#define SQR(n) ( ( n ) * ( n ) )
```

```
int num = 10;
```

```
int result = SQR(num);
```

```
cout << result << endl;
```

```
int result2 = SQR(5+5);
```

```
cout << result2 << endl;
```



输出一样吗？

带参宏定义

```
#define SQR(n) ( ( n ) * ( n ) )
```

在使用宏参数时，建议将所有的参数都是要 () 括起来。

像函数参数一样，在宏定义中的参数称为形式参数，在宏调用中的参数称为实际参数。

宏函数

C++ 中的宏函数，其实就是带参数的宏。

C++ 的宏函数跟普通函数类似，只是宏函数的参数没有类型。

宏函数 案例1

利用宏函数，实现判断字符是否是十进制数字！

宏函数 案例1 代码

利用宏函数，实现判断字符是否是十进制数字！

```
#define DIGIT(c) ((c) >= '0' && (c) <= '9')
```

```
char ch1 = 'a';
```

```
char ch2 = '2';
```

```
int d1 = DIGIT(ch1);
```

```
int d2 = DIGIT(ch2);
```

```
cout << d1 << endl;
```

```
cout << d2 << endl;
```

取消宏定义

定义了宏或者宏函数之后，不用了，可以使用 `undef` 取消定义宏和宏函数。

语法

```
#undef MACRO_NAME
```

取消宏定义 案例

```
#define PI 3.14
```

```
int main()
```

```
{
```

```
    cout << "PI = " << PI << endl;
```

```
    #undef PI
```

```
    cout << "PI = " << PI << endl;
```

```
    return 0;
```

```
}
```

//#undef 取消定义了宏，取消定义宏之后，不可以再次访问该宏

条件编译

有些时候，希望当满足某条件时对一组语句进行编译，而当条件不满足时则编译另一组语句。条件编译功能可按不同的条件去编译不同的程序部分，从而产生不同的目标代码文件。这对于程序的移植和调试是很有用的。

在 C++ 中，一般情况下，源程序中所有的行都参加编译。但有时希望对其中一部分内容只在满足一定条件才进行编译，也就是对一部分内容指定编译的条件，这就是“条件编译”。

条件编译作用

1.屏蔽跨平台差异

在大规模开发过程中，特别是跨平台和系统的软件里，可以在编译时通过条件编译设置编译环境。

2.包含程序功能模块

可以使用 `#ifdef` 实现包含程序的不同功能模块。

3.开关调试信息

调试程序时，常常希望输出一些所需的信息以便追踪程序的运行。而在调试完成后不再输出这些信息，可以通过条件编译控制。

4.防止头文件重复包含

头文件(.h)可以被头文件或 CPP 文件包含。由于头文件包含可以嵌套，CPP 文件就有可能多次包含同一个头文件；或者不同的 CPP 文件都包含同一个头文件，编译时就可能出现重复包含(重复定义)的问题。

条件编译 #ifdef

C++ 的 #ifdef 用于判断，如果一个标识符已被 #define 命令定义过，那么就编译该段代码，否则不编译。同时，#ifdef 还可以配合 #else 一起使用。

语法

```
#ifdef identity
```

```
    code1
```

```
#endif
```

如果标识符 identity 被定义过了，那么就编译代码 code1，否则就不编译。

条件编译 #ifdef

语法

```
#ifdef identity
```

```
    code1
```

```
#else
```

```
    code2
```

```
#endif
```

如果标识符 identity 被定义过了，那么就编译代码 code1，否则就编译代码 code2。

#ifdef案例

```
#include <iostream>
using namespace std;
int main()
{
    cout << "测试条件编译" << endl;
    #ifdef PRINT
        cout << "PRINT has defined" << endl;
    #endif
    return 0;
}
```

#ifdef案例

```
#include <iostream>
using namespace std;
int main()
{
    cout << "条件编译测试2" << endl;
    #ifdef PR
        cout << "PR has defined" << endl;
    #else
        cout << "PR not defined" << endl;
    #endif
    return 0;
}
```

条件编译 #ifndef

#ifndef正好和#ifdef反过来，用于判断，如果一个标识符没有被#define 命令定义过，那么就编译该段代码，否则不编译。

就是if not define的意思。

语法

```
#ifndef identity
```

```
    code1
```

```
#endif
```

如果标识符 identity 未被定义过，那么就编译代码 code1，否则就不编译。

条件编译 #ifndef

语法

```
#ifndef identity
```

```
    code1
```

```
#else
```

```
    code2
```

```
#endif
```

#ifndef 也可以配合 #else 一起使用，这里说明，如果标识符 identity 未被定义过，那么就编译代码 code1，否则就编译代码 code2。

#ifndef 案例

```
#include <iostream>
using namespace std;
int main()
{
    cout << "条件编译测试3" << endl;
    #ifndef PR
        cout << "PR not defined" << endl;
    #endif
    return 0;
}
```

#ifndef 案例

```
#include <iostream>
using namespace std;
int main()
{
    cout << "条件编译测试4" << endl;
    #ifndef PR
        cout << "PR not defined" << endl;
    #else
        cout << "PR has defined" << endl;
    #endif
    return 0;
}
```

条件编译 #if

C++ 的 #if 后面可以接常量表达式，如果常量表达式的值为真(非0)，则对一段程序进行编译，否则对另一段程序进行编译。因此可使程序在不同条件下，完成不同的功能。

语法1

```
#if expression  
    code1
```

```
#endif
```

语法2

```
#if expression  
    code1
```

```
#else
```

```
    code2
```

```
#endif
```

#if 案例

```
#include <iostream>
using namespace std;
int main()
{
    cout << "条件编译测试5" << endl;
    #define com 0
    #if com
        cout << "Start compile" << endl;
    #endif
    return 0;
}
```

#if 案例

```
#include <iostream>
using namespace std;
int main()
{
    cout << "条件编译测试5" << endl;
    #define com 0
    #if com
        cout << "Start compile" << endl;
    #else
        cout << " do not compile" << endl;
    #endif
    return 0;
}
```

条件编译 #elif

#elif可以跟在#if，#ifdef #ifndef的后面，但#else只能跟在最后面，用法和基本的条件语句类似。

#elif 案例

```
#include <iostream>
using namespace std;
int main(){

    cout << "条件编译测试" << endl;
    #define BY 0
    #define BA 0
    #if BY
        cout << "BY has define"<<endl;
    #elif BA
        cout << "BA is 1" <<endl;
    #endif

    return 0;
}
```

#error

#error 用于在编译期间给出一个错误信息，并终止程序的编译。同时，#error 后面的错误信息不需要使用双引号。

语法

#error error message

案例

#error compile error info

#error 案例

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    cout << "条件编译测试" << endl
```

```
    #error compile error here!
```

```
    return 0;
```

```
}
```

练习

利用宏函数，实现返回数组的元素个数。

练习代码

```
#define ARSIZE(arr) (sizeof((arr)) / sizeof((arr[0])))
```

```
int arr_num[5] = {1,2,3,4,5};
```

```
int len = ARSIZE(arr_num);
```

```
cout << len endl;
```

知识点总结

- | | |
|------------|----|
| 1. 预处理概述 | 了解 |
| 2. 常用预处理指令 | 熟悉 |
| 3. 宏定义 | 重点 |
| 4. 宏函数 | 重点 |
| 5. 取消宏定义 | 重点 |
| 6. 条件编译 | 重点 |

作业

定义 C++ 宏函数，实现求最大值和最小值

下节课预告

位运算